Software Requirements Specification (SRS)

Decentralised Voting Web3 Product Prototype — React.js Frontend, Express.js Backend & Solidity Smart Contract

Track: ShadowFox Blockchain Developer Internship (Advanced Level) | Standard: IEEE 830 / ISO 29148 | Version: 1.0

Document Attribute	Specification Value	Document Attribute	Specification Value
Project Name	Decentralised Voting Web3 dApp	**Internship Level**	Advanced Level (ShadowFox)
Frontend Stack	React.js 18 (Vite), Tailwind CSS, Ethers.js v6	**Backend Stack**	Node.js, Express.js REST API & Event Indexer
Blockchain Layer	Solidity `^0.8.20` (Hardhat / Sepolia Testnet)	**Wallet Standard**	EIP-1193 Browser Wallets (MetaMask)

1. Introduction & Product Rationale

1.1 Document Purpose & Scope

This Software Requirements Specification (SRS) establishes the complete functional, architectural, smart contract, API, and verification requirements for the **Decentralised Voting Web3 Prototype**. Designed for the **Advanced Level** track of the ShadowFox Blockchain Developer Internship, the system integrates an Ethereum Virtual Machine (EVM) Solidity smart contract with an Express.js indexing backend and a React.js single-page application to deliver an end-to-end, verifiable governance and election workflow.

1.2 Problem Statement & Why Blockchain Improves Voting Trust

Conventional web-based voting platforms store ballots in centralized relational or NoSQL databases controlled by a single administrator. Users must blindly trust that database administrators have not silently altered rows, injected synthetic ballots, or manipulated tally queries after voting closes. A blockchain-enabled architecture resolves these fundamental trust failures through three cryptographic guarantees:

- **Immutable Ballot Storage:** Every cast vote is signed by the voter's private key and committed to an append-only blockchain ledger via the `castVote` transaction, making silent retroactive modification computationally infeasible.
- **Protocol-Enforced One-Wallet-One-Vote:** The smart contract deterministically tracks participation using an on-chain nested mapping (`hasVoted[proposalId][msg.sender]`), rejecting duplicate votes at the EVM execution layer regardless of client or API state.
- **Public Auditability & Receipt Verification:** Any voter or observer can independently inspect emitted `VoteCast` events, transaction hashes, and live candidate tallies directly from the contract without relying on a privileged intermediary.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Technical Definition in Project Context
ABI	Application Binary Interface; JSON schema defining function signatures and events used by React and Express to encode/decode calls to `DecentralisedVoting.sol`.
EIP-1193	Ethereum Provider JavaScript API standard implemented by browser wallets (such as MetaMask) exposed at `window.ethereum`.
Hybrid Web3 Architecture	Design pattern where trust-critical state (eligibility, vote casting, tallying) executes strictly on-chain while rich metadata, search indexing, and event history caching run on an Express.js backend.

Term / Acronym	Technical Definition in Project Context
Metadata Content Hash	Deterministic SHA-256 digest computed from a proposal's rich-text description in Express.js and anchored on-chain inside the `Proposal` struct to prevent off-chain text tampering.
Transaction Lifecycle	The four user-visible stages of an on-chain action: (1) Wallet Signature Request, (2) Mempool Broadcast (`tx.hash`), (3) Block Confirmation (`tx.wait(1)`), and (4) UI State Synchronization.

2. Overall System Architecture & Stack Integration

2.1 Three-Tier Hybrid Web3 Architecture

To satisfy the Advanced Level requirement for a realistic, usable Web3 product prototype rather than an isolated contract deployment demo, the system separates responsibilities across three cohesive tiers while preserving strict on-chain trust boundaries:

System Tier	Technology Components	Primary Responsibilities	Security & Trust Boundary
1. Presentation & Signer Layer	React.js 18, Vite, Tailwind CSS, Ethers.js v6 (`BrowserProvider`)	Manages wallet connection, network validation, proposal dashboard rendering, interactive transaction progress modals, and client-side transaction signing.	Private keys remain strictly inside the user's wallet extension; signs state-mutating calls directly.
2. Application & Indexing Layer	Node.js, Express.js, Ethers.js (`JsonRpcProvider`), JSON/SQLite Cache	Stores extended proposal descriptions, generates SHA-256 content hashes, indexes `ProposalCreated` and `VoteCast` events, and serves fast REST responses.	Untrusted performance and UX layer; cannot forge votes or override on-chain tallies.
3. Consensus & Execution Layer	Solidity `^0.8.20`, Hardhat Network / Ethereum Sepolia Testnet	Enforces proposal deadlines, verifies voter eligibility, prevents double voting, increments candidate vote counts atomically, and emits audit events.	Authoritative, tamper-resistant source of truth for all election outcomes.

2.2 End-to-End Workflow Interaction

1. **Discovery & Read Flow:** When a user opens the React frontend, the app fetches the enriched proposal list and recent audit logs from the Express.js API (`GET /api/proposals`) while simultaneously verifying live vote counts and connected wallet receipts directly from the Solidity smart contract.
2. **Proposal Creation Flow:** An authorized creator submits a proposal title, detailed manifesto, candidate/option list, and voting duration in React. The frontend sends the manifesto to `POST /api/proposals/metadata` to receive a deterministic `metadataHash`, then prompts MetaMask to sign `createProposal(title, metadataHash, optionNames, duration)` on-chain.
3. **Vote Execution Flow:** A connected voter selects a candidate/option on an active proposal and clicks **Cast Vote**. React invokes `contract.castVote(proposalId, optionId)` via the MetaMask signer, displays real-time transaction progress (Pending → Confirmed), and triggers an Express indexer sync so both on-chain state and audit logs update immediately.

2.3 User Classes and Permissions

- **Unconnected Visitor (Read-Only Observer):** Can browse active and past proposals, inspect candidate vote distributions, and view on-chain audit transaction hashes via the Express read-only provider, with clear prompts to connect a wallet before voting.
- **Connected Voter (Standard Web3 User):** Connects an EVM wallet on the target network, casts exactly one vote per active proposal, tracks live transaction confirmations, and views a permanent on-chain voter receipt badge showing

their verified selection.

- **Election Administrator / Proposal Creator:** The contract deployer (or any wallet when public proposal mode is enabled) who can create new voting proposals, configure voting durations, and trigger `finalizeProposal` once the deadline passes.

---

3. Smart Contract Specification (`DecentralisedVoting.sol`)

3.1 On-Chain State Variables & Struct Definitions

The Solidity smart contract is compiled under `pragma solidity ^0.8.20;` to leverage native arithmetic overflow/underflow protection and custom revert errors for gas-efficient validation feedback:

- `address public admin;` — Stores the deployer address with administrative privileges to manage election settings.
- `bool public allowPublicProposals;` — Configurable flag allowing either admin-only or community-wide proposal creation during demonstrations.
- `uint256 public proposalCount;` — Auto-incrementing counter serving as the unique primary key for each created proposal.
- `struct Option { uint256 id; string name; uint256 voteCount; }` — Represents an individual ballot choice or candidate within a proposal.
- `struct Proposal { uint256 id; string title; string metadataHash; address creator; uint256 startTime; uint256 endTime; uint256 totalVotes; bool finalized; uint256 winningOptionId; }` — Core proposal record stored in `mapping(uint256 => Proposal) public proposals`.
- `mapping(uint256 => Option[]) private proposalOptions;` — Stores the list of selectable options for each proposal ID.
- `mapping(uint256 => mapping(address => bool)) public hasVoted;` — Tracks whether a wallet address has already cast a ballot on a specific proposal.
- `mapping(uint256 => mapping(address => uint256)) public voterChoice;` — Records the option index selected by each voter so the React UI can display an authenticated voter receipt.

3.2 Smart Contract Function Specification Matrix

Function Signature	Mutability	On-Chain Preconditions (`require`)	State Transitions & Emitted Events
`createProposal(string title, string metadataHash, string[] optionNames, uint256 durationSeconds)`	`external` (Write)	Caller is `admin` or `allowPublicProposals == true`; `bytes(title).length > 0`; `optionNames.length >= 2 && optionNames.length <= 10`; `durationSeconds >= 60`.	Increments `proposalCount`; writes `Proposal` struct with `endTime = block.timestamp + durationSeconds`; populates `proposalOptions`; emits `ProposalCreated`.
`castVote(uint256 proposalId, uint256 optionIndex)`	`external` (Write)	`proposalId > 0 && proposalId <= proposalCount`; `block.timestamp <= proposal.endTime`; `!hasVoted[proposalId][msg.sender]`; `optionIndex < proposalOptions[proposalId].length`.	Sets `hasVoted[proposalId][msg.sender] = true`; stores `voterChoice`; increments option `voteCount` and `totalVotes`; emits `VoteCast(proposalId, msg.sender, optionIndex, block.timestamp)`.
`finalizeProposal(uint256 proposalId)`	`external` (Write)	Proposal exists; `block.timestamp > proposal.endTime`; `!proposal.finalized`.	Iterates options to identify highest `voteCount`; sets `finalized = true` and `winningOptionId`; emits `ProposalFinalized(proposalId, winningOptionId, totalVotes)`.

Function Signature	Mutability	On-Chain Preconditions (`require`)	State Transitions & Emitted Events
`getProposal(uint256 proposalId)`	`external view` (Read)	`proposalId > 0 && proposalId <= proposalCount`.	Returns tuple of **`Proposal`** metadata, computed status (**`Active`**, **`Ended`**, **`Finalized`**), and full **`Option[]`** array with live tallies.
`getVoterReceipt(uint256 proposalId, address voter)`	`external view` (Read)	Proposal exists.	Returns **`(bool voted, uint256 selectedOptionIndex)`** for instant frontend verification of the connected user's voting status.

3.3 On-Chain Events & Custom Errors

- `event ProposalCreated(uint256 indexed proposalId, string title, address indexed creator, uint256 startTime, uint256 endTime, string metadataHash);`
- `event VoteCast(uint256 indexed proposalId, address indexed voter, uint256 indexed optionIndex, uint256 timestamp);`
- `event ProposalFinalized(uint256 indexed proposalId, uint256 winningOptionIndex, uint256 totalVotes);`
- **Custom Errors:** `UnauthorizedCreator()`, `InvalidProposalId()`, `VotingWindowClosed()`, `VotingStillActive()`, `AlreadyVotedOnProposal()`, and `InvalidOptionIndex()`.

---

4. Detailed Functional Requirements (FR)

4.1 Wallet Connection & State Management (React Frontend)

- **FR-1.1 (EIP-1193 Provider Detection):** On initial render, the React application shall inspect **`window.ethereum`**. If no wallet extension is installed, the UI shall display a clear banner linking to MetaMask installation while keeping read-only proposal browsing enabled via the Express backend.
- **FR-1.2 (Explicit Wallet Connection Flow):** Clicking **Connect Wallet** shall invoke **`eth_requestAccounts`**, store the active signer address in React context, fetch the wallet's native ETH balance for gas readiness checks, and load the user's voting receipts across all proposals.
- **FR-1.3 (Dynamic Account & Chain Listeners):** The frontend shall bind listeners to **`window.ethereum.on('accountsChanged')`** and **`window.ethereum.on('chainChanged')`**. Switching accounts in MetaMask must immediately re-evaluate whether the newly selected wallet has voted on each proposal without requiring a page refresh.
- **FR-1.4 (Target Network Enforcement):** If the connected wallet's chain ID does not match the configured deployment network (Hardhat **`31337`** or Sepolia **`11155111`**), all write buttons (**Create Proposal**, **Cast Vote**) shall be locked and replaced with a one-click **Switch to Target Network** button calling **`wallet_switchEthereumChain`**.

4.2 Proposal Creation & Hybrid Metadata Sync

- **FR-2.1 (Validated Proposal Form):** The React creation modal shall validate that the proposal title is between 5 and 120 characters, the description provides clear context, at least 2 distinct voting options are entered, and a valid duration (e.g., 1 hour, 24 hours, 7 days, or custom demo duration of 5 minutes) is selected.
- **FR-2.2 (Off-Chain Metadata Hashing):** Prior to invoking the smart contract, the React app shall send the extended proposal description and category tag to the Express.js endpoint **`POST /api/proposals/metadata`**, which computes a deterministic SHA-256 **`metadataHash`** and persists the text record.
- **FR-2.3 (On-Chain Proposal Registration):** The frontend shall pass the returned **`metadataHash`** into **`contract.createProposal(...)`**. Once confirmed on-chain, the Express event listener captures **`ProposalCreated`** and links the off-chain metadata to the newly assigned on-chain **`proposalId`**.

4.3 Vote Casting, Transaction Handling & User Feedback

- **FR-3.1 (Pre-Vote Eligibility & Receipt Display):** For every proposal card, the frontend shall query `getVoterReceipt(proposalId, connectedAddress)`. If `voted == true`, the UI shall display a verified badge (e.g., *"You voted for Option #2 — Verified On-Chain"*) and disable the vote submission button.
- **FR-3.2 (Four-Stage Transaction Feedback Stepper):** To fulfill the ShadowFox requirement that transaction flows remain visible and understandable, submitting a vote or creating a proposal shall display a live progress indicator across four distinct states:
  1. **Step 1 — Waiting for Wallet Signature:** Instructs the user to confirm the transaction in their MetaMask popup.
  2. **Step 2 — Broadcasting & Mining in Mempool:** Displays the generated transaction hash (`0x...`) immediately after signature along with an elapsed timer while awaiting `tx.wait(1)`.
  3. **Step 3 — Confirmed on Blockchain:** Displays the confirmed block number, gas consumed, and a direct link to verify the transaction.
  4. **Step 4 — State Synchronized:** Automatically re-queries contract state and updates candidate progress bars and total vote counters in place.
- **FR-3.3 (Human-Readable Error Decoding):** If a transaction fails or is cancelled, the frontend error handler shall translate low-level RPC/EVM errors into actionable user feedback (e.g., mapping code `4001` to *"Transaction signature was cancelled in your wallet"* and custom error `AlreadyVotedOnProposal` to *"This wallet has already cast a vote on this proposal"*).

---

5. External Interface & Express.js API Specification

5.1 Express.js Backend REST API Endpoints

The Express.js server acts as a high-speed read cache, off-chain metadata store, and contract event indexer. All endpoints return structured JSON with CORS enabled for the React Vite frontend:

Method	Endpoint Route	Request Parameters / Body	Response Payload & Behavior
`GET`	`/api/health`	None	Returns `{ status: "healthy", networkChainId, contractAddress, latestSyncedBlock }` to verify RPC connectivity.
`GET`	`/api/proposals`	Query: `?status=all|active|ended`, `?search=keyword`	Merges on-chain `Proposal` structs and live option tallies with cached off-chain descriptions and categories.
`GET`	`/api/proposals/:id`	Path param: `:id` (uint256 proposal ID)	Returns complete proposal detail, option percentages, SHA-256 integrity check status, and chronological `VoteCast` audit logs.
`POST`	`/api/proposals/metadata`	JSON: `{ title, description, category, creator }`	Computes SHA-256 hash of payload, saves metadata record, and returns `{ metadataHash, createdAt }` for smart contract input.
`GET`	`/api/audit/events`	Query: `?proposalId=1&limit=50`	Queries indexed `ProposalCreated`, `VoteCast`, and `ProposalFinalized` contract events with block numbers and tx hashes.

Method	Endpoint Route	Request Parameters / Body	Response Payload & Behavior
`POST`	`/api/sync`	JSON: `{ txHash, proposalId }`	Triggered by React after `tx.wait(1)` to force an immediate backend re-index of the latest block state.

5.2 React Frontend Component Architecture

- **`WalletProvider.jsx` (Web3 Context):** Centralizes Ethers.js `BrowserProvider`, `JsonRpcSigner`, contract instance binding, account address, chain ID, and automatic event listeners.
- **`HeaderNavbar.jsx`:** Renders application title, live network badge (e.g., *Sepolia Testnet* or *Hardhat Local*), truncated wallet address, ETH balance, and connect/disconnect actions.
- **`ProposalDashboard.jsx`:** Displays summary metrics (Total Proposals, Active Polls, Total On-Chain Votes Cast), status filter tabs, search bar, and responsive proposal cards.
- **`VotingModal.jsx`:** Displays proposal description, SHA-256 metadata verification badge, candidate progress bars with live vote shares, radio selection inputs, and the 4-stage transaction progress stepper.
- **`AuditExplorer.jsx`:** Renders a verifiable table of on-chain `VoteCast` events showing voter address, proposal ID, option voted, block number, and transaction hash.

---

6. Non-Functional Requirements (NFR) & Security Controls

Requirement ID	Category	Specification & Implementation Mechanism
NFR-SEC-01	Sybil & Double-Vote Resistance	The smart contract must enforce `!hasVoted[proposalId][msg.sender]` before mutating state (Checks-Effects-Interactions pattern), guaranteeing zero duplicate ballots per address.
NFR-SEC-02	Off-Chain Tamper Detection	The React frontend must recompute the SHA-256 hash of the description returned by Express and compare it against the on-chain `metadataHash`, flagging any mismatch.
NFR-SEC-03	Non-Custodial Key Safety	Neither the React frontend nor the Express backend shall ever request, transmit, or store user private keys or seed phrases; all write transactions are signed locally in MetaMask.
NFR-REL-01	Zero-Downtime Chain Fallback	If the Express.js backend is unavailable, the React application must automatically fall back to direct Ethers.js contract reads via `window.ethereum` so voting remains 100% functional.
NFR-PERF-01	UI State Refresh Latency	Following block confirmation (`tx.wait(1)`), updated vote counts and voter receipt status must render in the React UI within 1.5 seconds without a browser reload.

---

7. Verification Matrix & ShadowFox Submission Mapping

7.1 End-to-End Acceptance Test Suite

Test ID	Test Scenario	Test Input & Action	Expected System & On-Chain Result
TC-01	Unconnected Visitor Flow	Open React app without connecting MetaMask wallet.	Proposals and live tallies load via Express API; vote buttons are disabled with *"Connect Wallet to Vote"* prompt.

Test ID	Test Scenario	Test Input & Action	Expected System & On-Chain Result
TC-02	Wallet & Network Handshake	Click *Connect Wallet* and test on wrong chain vs. target chain.	Header displays truncated address; wrong chain shows *Switch Network* prompt; target chain unlocks voting.
TC-03	Hybrid Proposal Creation	Submit new proposal with 3 candidates and sign in MetaMask.	Express returns **`metadataHash`**; contract emits **`ProposalCreated`**; new poll appears in Active tab.
TC-04	Valid Vote & Progress Stepper	Select Candidate #1, click *Cast Vote*, and approve transaction.	Stepper advances through Signature → Pending (**`txHash`**) → Confirmed; candidate tally increments by 1.
TC-05	Duplicate Vote Rejection	Attempt to vote a second time on the same proposal with the same wallet.	UI locks voting controls with verified receipt badge; direct contract call reverts with **`AlreadyVotedOnProposal`**.
TC-06	Deadline Expiry & Finalization	Attempt to vote after **`block.timestamp > endTime`**, then click *Finalize*.	**`castVote`** reverts with **`VotingWindowClosed`**; **`finalizeProposal`** records winner and emits **`ProposalFinalized`**.

7.2 GitHub Repository Structure & Mandatory Video Walkthrough Plan

To satisfy the ShadowFox Advanced Level submission process (pages 5–6 of the internship task list), the project repository and mandatory 3–5 minute Google Drive explanation video shall follow this structure:

- **GitHub Monorepo Layout:**
  - **`/contracts`** — Contains **`DecentralisedVoting.sol`**, Hardhat configuration (**`hardhat.config.js`**), deployment & seed script (**`scripts/deploy.js`**), and automated Mocha/Chai contract tests.
  - **`/backend`** — Contains Express.js application (**`server.js`**), Ethers.js event indexer, SHA-256 metadata hashing controller, and REST API routes.
  - **`/frontend`** — Contains React (Vite) source code, **`WalletProvider`** context, transaction stepper component, and compiled contract ABI JSON.
  - **`/docs`** — Contains this Software Requirements Specification (SRS), architecture diagrams, and deployment instructions.
- **Mandatory 3–5 Minute Demo Video Script (Shared via Google Drive Link):**
  - **0:00 – 0:45 (Use Case & Blockchain Value):** Explain why centralised voting suffers from silent database tampering and how on-chain state guarantees trustless verification.
  - **0:45 – 1:45 (Smart Contract & Architecture Walkthrough):** Show **`DecentralisedVoting.sol`** structs, mappings, and modifiers, plus how React, Express, and Solidity communicate.
  - **1:45 – 3:30 (Live End-to-End Product Demo):** Connect MetaMask in the React UI, create a proposal, cast a vote, highlight the real-time transaction progress modal (Pending → Confirmed), and show the live tally update.
  - **3:30 – 4:30 (Tamper Resistance & Auditability):** Show the on-chain voter receipt blocking duplicate votes, switch to a second MetaMask account to cast another vote, and inspect the live **`VoteCast`** event audit log.